

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 10 – PENCARIAN NILAI MAX MIN



DEVIA DIAN INDRIYANTI

109082500150

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pencarian nilai ekstrem, yaitu nilai maksimum dan minimum, merupakan algoritma fundamental dalam pengolahan data menggunakan struktur data *array*. Algoritma ini bertujuan untuk mengidentifikasi elemen dengan nilai tertinggi atau terendah di dalam sekumpulan data yang tersimpan secara sekuensial.

Secara teknis, proses ini dilakukan dengan cara menginisialisasi sebuah variabel penampung sementara menggunakan elemen pertama dari *array* sebagai nilai acuan. Selanjutnya, program akan melakukan iterasi atau perulangan melalui seluruh elemen *array*. Pada setiap iterasi, dilakukan perbandingan menggunakan struktur kontrol kondisi (if):

- **Nilai Maksimum:** Jika elemen yang sedang diperiksa lebih besar dari nilai di variabel penampung, maka variabel tersebut akan diperbarui dengan nilai elemen yang baru.
- **Nilai Minimum:** Jika elemen yang sedang diperiksa lebih kecil dari nilai di variabel penampung, maka variabel tersebut akan diperbarui dengan nilai elemen yang baru.

Algoritma ini tidak hanya dapat diterapkan pada tipe data primitif seperti *integer* atau *float64*, tetapi juga pada tipe data bentukan seperti *struct*. Selain mencari nilainya, algoritma ini juga sering digunakan untuk melacak posisi atau indeks dari nilai tersebut di dalam memori.

B. Guided

1. Guided 1 – Modul 10

```
package main
import "fmt"

func nilaiTerbesar(array []int) (int, int) {
    var idx int
    var nilaiTerbesar int = array[0]
    var idxDitemukan int = 0
    for idx = 1; idx < len(array); idx++ {
        if array[idx] > nilaiTerbesar {
            nilaiTerbesar = array[idx]
            idxDitemukan = idx
        }
    }
    return nilaiTerbesar, idxDitemukan
}
```

```

func nilaiTerkecil(array []int) (int, int) {
    var idx int
    var nilaiTerkecil int = array[0]
    var idxDitemukan int = 0
    for idx = 1; idx < len(array); idx++ {
        if array[idx] < nilaiTerkecil {
            nilaiTerkecil = array[idx]
            idxDitemukan = idx
        }
    }
    return nilaiTerkecil, idxDitemukan
}

func main() {
    var arrAngka [5]int = [5]int{45, 23, 98, 66, 12}

    var nilaiMaks, idxNilaiMaks int = nilaiTerbesar(arrAngka[:])
    var nilaiMin, idxNilaiMin int = nilaiTerkecil(arrAngka[:])

    fmt.Printf("Nilai terkecil : %d ditemukan pada index ke-%d",
nilaiMin, idxNilaiMin)
    fmt.Println()
    fmt.Printf("Nilai terbesar : %d ditemukan pada index ke-%d",
nilaiMaks, idxNilaiMaks)
}

```

Output :

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devia-Dian-Indriyanti\
Week 9-Modul 10\Guided\Guided 1\contoh1.go"
Nilai terkecil : 12 ditemukan pada index ke-4
Nilai terbesar : 98 ditemukan pada index ke-2
PS C:\109082500150_Devia-Dian-Indriyanti>

```

2. Guided 2 – Modul 10

```

package main
import "fmt"

type mahasiswa struct {

```

```

    nama string
    nim string
    IPK float64
}

type arrMahasiswa [3]mahasiswa

func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
    var mhsTerbesar mahasiswa = array[0]
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK > mhsTerbesar.IPK {
            mhsTerbesar = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerbesar, idxDitemukan
}

func IPKTerkecil(array []mahasiswa) (mahasiswa, int) {
    var mhsTerkecil mahasiswa = array[0]
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK < mhsTerkecil.IPK {
            mhsTerkecil = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerkecil, idxDitemukan
}

func main() {
    var arrMhs arrMahasiswa
    var i, j int
    for i = 0; i < len(arrMhs); i++ {
        fmt.Println("DATA MAHASISWA INDEKS KE-", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&arrMhs[i].nama)
        fmt.Print("Masukkan NIM :")
        fmt.Scan(&arrMhs[i].nim)
    }
}

```

```

        fmt.Println("Masukkan IPK : ")
        fmt.Scan(&arrMhs[i].IPK)
        fmt.Println("=====")
    }

    fmt.Println()
    for j = 0; j < len(arrMhs); j++ {
        fmt.Println("DATA MAHASISWA INDEKS KE-", j)
        fmt.Println("Nama : ", arrMhs[j].nama)
        fmt.Println("NIM : ", arrMhs[j].nim)
        fmt.Println("IPK : ", arrMhs[j].IPK)
        fmt.Println("=====")
    }
    fmt.Println()

    var mhsMaks, mhsMin mahasiswa
    var idxMaks, idxMin int

    mhsMaks, idxMaks = IPKTerbesar(arrMhs[:])
    mhsMin, idxMin = IPKTerkecil(arrMhs[:])

    fmt.Println("=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===")
    fmt.Printf("IPK Terbesar : %.2f, atas nama %s dengan NIM %s,
ditemukan pada indeks ke-%d", mhsMaks.IPK, mhsMaks.nama, mhsMaks.nim,
idxMaks)
    fmt.Printf("IPK Terkecil : %.2f, atas nama %s dengan NIM %s,
ditemukan pada indeks ke-%d", mhsMin.IPK, mhsMin.nama, mhsMin.nim,
idxMin)
}

```

Output :

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devia-Dian-Indriyanti
Week 9-Modul 10\Guided\Guided 2\contoh2.go"
DATA MAHASISWA INDEKS KE- 0
Masukkan nama : jarwo
Masukkan NIM : 109082667483
Masukkan IPK : 3
=====
DATA MAHASISWA INDEKS KE- 1
Masukkan nama : gibran
Masukkan NIM : 262783642
Masukkan IPK : 4
=====
DATA MAHASISWA INDEKS KE- 2
Masukkan nama : sopo
Masukkan NIM : 962436819
Masukkan IPK : 2
=====
DATA MAHASISWA INDEKS KE- 0
Nama : jarwo
NIM : 109082667483
IPK : 3
=====
DATA MAHASISWA INDEKS KE- 1
Nama : gibran
NIM : 262783642
IPK : 4
=====
DATA MAHASISWA INDEKS KE- 2
Nama : sopo
NIM : 962436819
IPK : 2
=====
=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===
IPK Terbesar : 4.00, atas nama gibran dengan NIM 262783642, ditemukan pada indeks ke-1
IPK Terkecil : 2.00, atas nama sopo dengan NIM 962436819, ditemukan pada indeks ke-2
PS C:\109082500150_Devia-Dian-Indriyanti>

```

C. Unguided

1. Soal Unguided 1

Sebuah program digunakan untuk mendata berat anak kelinci yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat anak kelinci yang akan dijual.

Masukan terdiri dari sekumpulan bilangan, yang mana bilangan pertama adalah bilangan bulat N yang menyatakan banyaknya anak kelinci yang akan ditimbang beratnya. Selanjutnya N bilangan riil berikutnya adalah berat dari anak kelinci yang akan dijual.

Jawaban :

```
package main
import "fmt"

func main() {
    var N int
    var berat [1000] float64
    fmt.Scan(&N)

    for i := 0; i < N; i++ {
        fmt.Scan(&berat[i])
    }

    min := berat[0]
    max := berat[0]

    for i := 1; i < N; i++ {
        if berat[i] < min {
            min = berat[i]
        }
        if berat[i] > max {
            max = berat[i]
        }
    }
    fmt.Println(min, max)
}
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devia-Dian-Indriyanti\Week 9-Modul 10\Unguided\Unguided 1\soal1.go"
5
1.2 0.8 2.5 1.0 3.1
0.8 3.1
PS C:\109082500150_Devia-Dian-Indriyanti> |
```

Penjelasan :

Program melakukan pembacaan jumlah data N dan menyimpannya ke dalam array. Nilai elemen pertama ditetapkan sebagai acuan awal untuk variabel min dan max. Selanjutnya, program melakukan iterasi dari elemen kedua hingga terakhir untuk membandingkan setiap data. Jika ditemukan nilai yang lebih kecil dari min atau lebih besar dari max, maka variabel terkait akan diperbarui dengan nilai baru tersebut. Pada tahap akhir, program menampilkan nilai minimum dan maksimum yang telah ditemukan.

2. Soal Unguided 2

Sebuah program digunakan untuk menentukan tarif ikan yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat ikan yang akan dijual.

Masukan terdiri dari dua baris, yang mana baris pertama terdiri dari dua bilangan bulat x dan y. Bilangan x menyatakan banyaknya ikan yang akan dijual, sedangkan y adalah banyaknya ikan yang akan dimasukkan ke dalam wadah. Baris kedua terdiri dari sejumlah x bilangan riil yang menyatakan banyaknya ikan yang akan dijual.

Jawaban :

```
package main
import "fmt"

func main() {
    var x, y int
    fmt.Scan(&x, &y)
    berat := make([]float64, x)

    for i := 0; i < x; i++ {
        fmt.Scan(&berat[i])
    }
}
```

```

    }

    hasil := make([]float64, 0)
    totalAll := 0.0

    for i := 0; i < x; i += y {
        total := 0.0

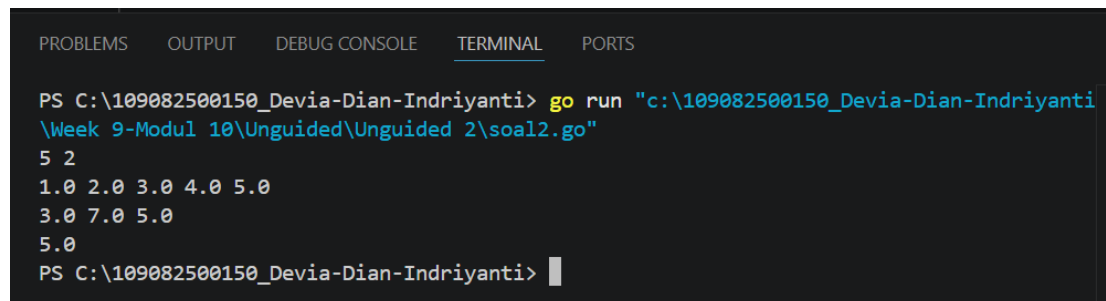
        for j := i; j < i+y && j < x; j++ {
            total += berat[j]
        }
        hasil = append(hasil, total)
        totalAll += total
    }

    for i := 0; i < len(hasil); i++ {
        fmt.Printf("%.1f ", hasil[i])
    }
    fmt.Println()

    rata := totalAll / float64(len(hasil))
    fmt.Printf("%.1f\n", rata)
}

```

Output :



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devia-Dian-Indriyanti
\Week 9-Modul 10\Unguided\Unguided 2\soal2.go"
5 2
1.0 2.0 3.0 4.0 5.0
3.0 7.0 5.0
5.0
PS C:\109082500150_Devia-Dian-Indriyanti>

```

Penjelasan :

Program menginput jumlah ikan (x) serta kapasitas wadah (y), kemudian menyimpan seluruh data berat ikan ke dalam array. Data tersebut diproses secara berkelompok berdasarkan nilai y menggunakan struktur perulangan dengan interval tertentu (i += y). Di dalam setiap kelompok, terdapat perulangan sekunder untuk menghitung total berat per wadah. Setelah seluruh kelompok terhitung, program menghitung rata-rata dari total berat semua wadah dan menampilkan hasilnya beserta total berat masing-masing wadah ke layar.

3. Soal Unguided 3

Pos Pelayanan Terpadu (posyandu) sebagai tempat pelayanan kesehatan perlu mencatat data berat balita (dalam kg). Petugas akan memasukkan data tersebut ke dalam array. Dari data yang diperoleh akan dicari berat balita terkecil, terbesar, dan reratanya.

Jawaban :

```
package main
import "fmt"

type arrBalita [100]float64
func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax *float64) {
    *bMin = arrBerat[0]
    *bMax = arrBerat[0]

    for i := 1; i < n; i++ {
        if arrBerat[i] < *bMin {
            *bMin = arrBerat[i]
        }
        if arrBerat[i] > *bMax {
            *bMax = arrBerat[i]
        }
    }
}

func rerata(arrBerat arrBalita, n int) float64 {
    total := 0.0
    for i := 0; i < n; i++ {
        total += arrBerat[i]
    }
    return total / float64(n)
}

func main() {
    var data arrBalita
    var n int
    var min, max float64

    fmt.Print("masukkan banyak berat balita: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Printf("masukkan berat balita ke-%d: ", i+1)
```

```

        fmt.Scan(&data[i])
    }
    hitungMinMax(data, n, &min, &max)
    rata := rerata(data, n)

    fmt.Printf("berat balita minimum: %.2f kg\n", min)
    fmt.Printf("berat balita maksimum: %.2f kg\n", max)
    fmt.Printf("rata-rata berat balita: %.2f kg\n", rata)
}

```

Output :

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devia-Dian-Indriyanti\Week 9-Modul 10\Unguided\Unguided 3\soal3.go"
masukkan banyak berat balita: 4
masukkan berat balita ke-1: 5.3
masukkan berat balita ke-2: 6.2
masukkan berat balita ke-3: 4.1
masukkan berat balita ke-4: 9.9
berat balita minimum: 4.10 kg
berat balita maksimum: 9.90 kg
rata-rata berat balita: 6.38 kg
PS C:\109082500150_Devia-Dian-Indriyanti>

```

Penjelasan :

Program ini digunakan untuk mengolah data berat balita yang disimpan dalam array. Pertama, user memasukkan jumlah data lalu menginput tiap berat balita. Setelah itu, prosedur `hitungMinMax` dipakai untuk mencari nilai terkecil dan terbesar dengan cara membandingkan setiap elemen dalam array. Sedangkan fungsi `rerata` digunakan untuk menghitung rata-rata dengan menjumlahkan semua data lalu dibagi jumlahnya. Jadi, program ini memanfaatkan perulangan untuk memproses data satu per satu dan menggunakan prosedur serta fungsi agar program lebih terstruktur dan mudah dipahami.

D. Kesimpulan

Berdasarkan hasil praktikum pada Modul 10, dapat ditarik beberapa kesimpulan sebagai berikut:

- Pencarian nilai maksimum dan minimum pada *array* dilakukan dengan memanfaatkan mekanisme perulangan (*looping*) untuk memeriksa setiap elemen secara menyeluruh.

- Implementasi algoritma menggunakan fungsi dan prosedur (seperti `hitungMinMax` atau `IPKTerbesar`) membuat struktur kode program menjadi lebih modular, sistematis, dan memudahkan proses pemeliharaan kode.
- Algoritma pencarian nilai ekstrem ini sangat fleksibel karena dapat diterapkan pada berbagai skenario data, mulai dari pengolahan data numerik sederhana hingga pengolahan data kompleks dalam bentuk *array of struct*.
- Selain mendapatkan nilai akhir, penggunaan variabel tambahan untuk menyimpan indeks memungkinkan program untuk mengetahui lokasi tepat di mana data tersebut ditemukan.